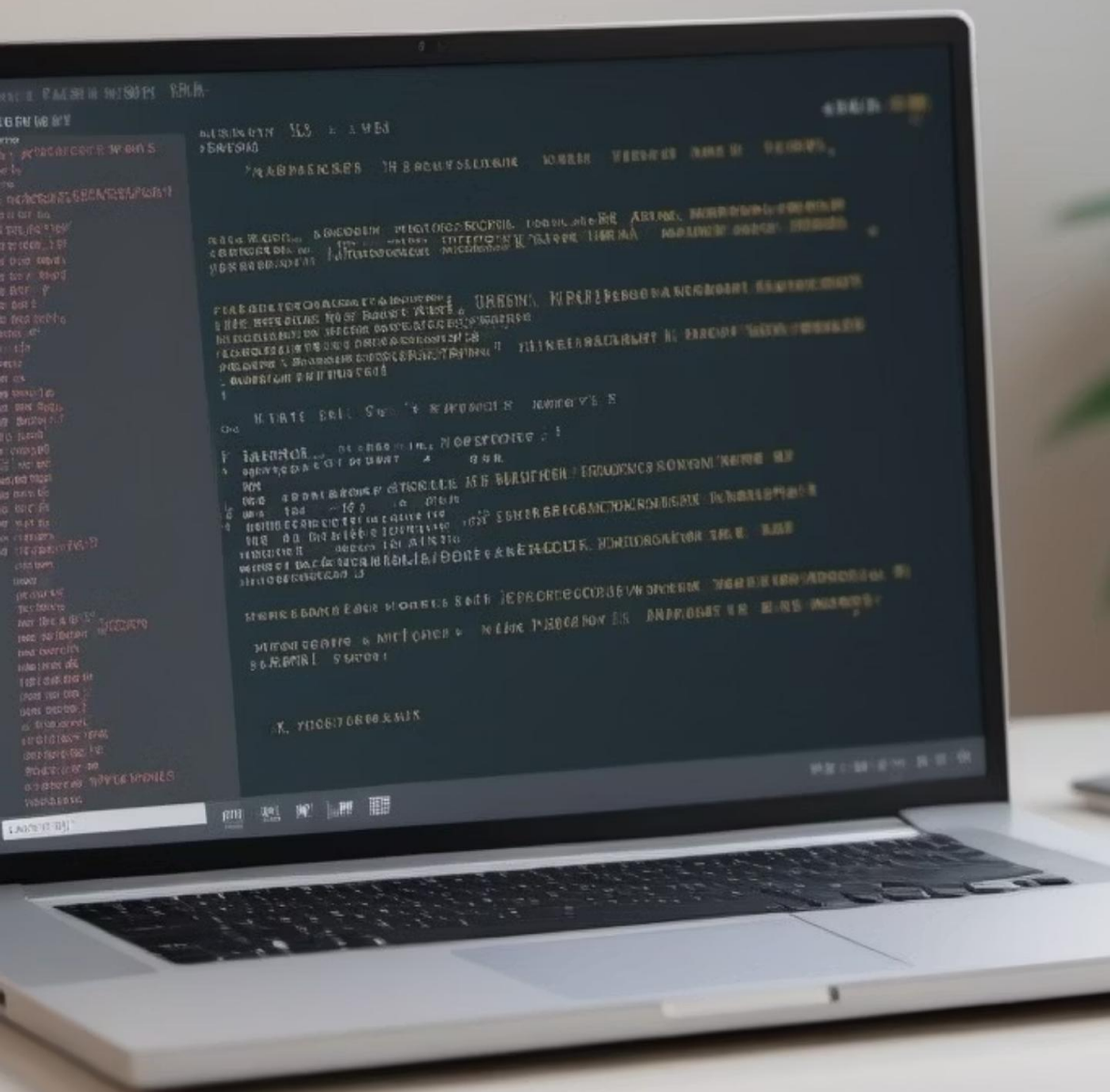


Codificação Para Front-end

17

2026

SENAI - CRTI



Objetos, Arrays e JSON

Estruturando os dados do nosso e-commerce com JavaScript para renderização dinâmica na interface.

O Desafio do E-Commerce

O problema real

Nosso e-commerce está crescendo. Precisamos exibir **dezenas de produtos** de forma organizada, escalável e pronta para ser renderizada dinamicamente na interface.

Até agora, usamos variáveis simples. Mas como gerenciar nome, preço, categoria e imagem de 50 produtos ao mesmo tempo?

Nossa missão nesta aula

- Estruturar dados com **objetos**
- Agrupar com **arrays de objetos**
- Entender o formato **JSON**
- Usar **destructuring e spread**

Capacidades a Desenvolver



Objetos JS

Criar e manipular objetos com propriedades e métodos em JavaScript.



Destructuring

Extrair dados de objetos e arrays de forma elegante.



Arrays de Objetos

Trabalhar com coleções de dados estruturados em listas.



Spread Operator

Copiar e expandir listas e objetos com ...



JSON

Compreender o formato padrão de troca de dados na web.

Retomada: Lógica com JavaScript



O que já sabemos

Nas aulas anteriores, aprendemos a usar variáveis, condicionais e funções para controlar o comportamento da página. Exemplo simples de produto com variáveis separadas:

```
let nomeProduto = "Tênis Esportivo";  
let precoProduto = 199.90;  
let categoriaProduto = "Calçados";  
let imagemProduto = "tenis.jpg";
```

O problema desta abordagem

Funciona para **um produto**. Mas e quando temos 50 produtos? Teríamos **200 variáveis** soltas, sem organização, difíceis de manter e impossíveis de escalar. É hora de evoluir!

⚠ Variáveis isoladas não escalam. Precisamos de uma estrutura de dados mais poderosa.

O Que São Objetos em JavaScript?

Um **objeto** é uma estrutura que agrupa dados relacionados (propriedades) e comportamentos (métodos) em uma única entidade. Pense nele como uma "ficha de produto".

Sintaxe básica

```
const produto = {  
  nome: "Tênis Esportivo",  
  preco: 199.90,  
  categoria: "Calçados",  
  imagem: "tenis.jpg",  
  emEstoque: true  
};
```

Acesse propriedades com `produto.nome` ou `produto["preco"]`.

Por que usar objetos?

Organização

Todos os dados do produto em um só lugar.

Legibilidade

Código mais claro e fácil de entender.

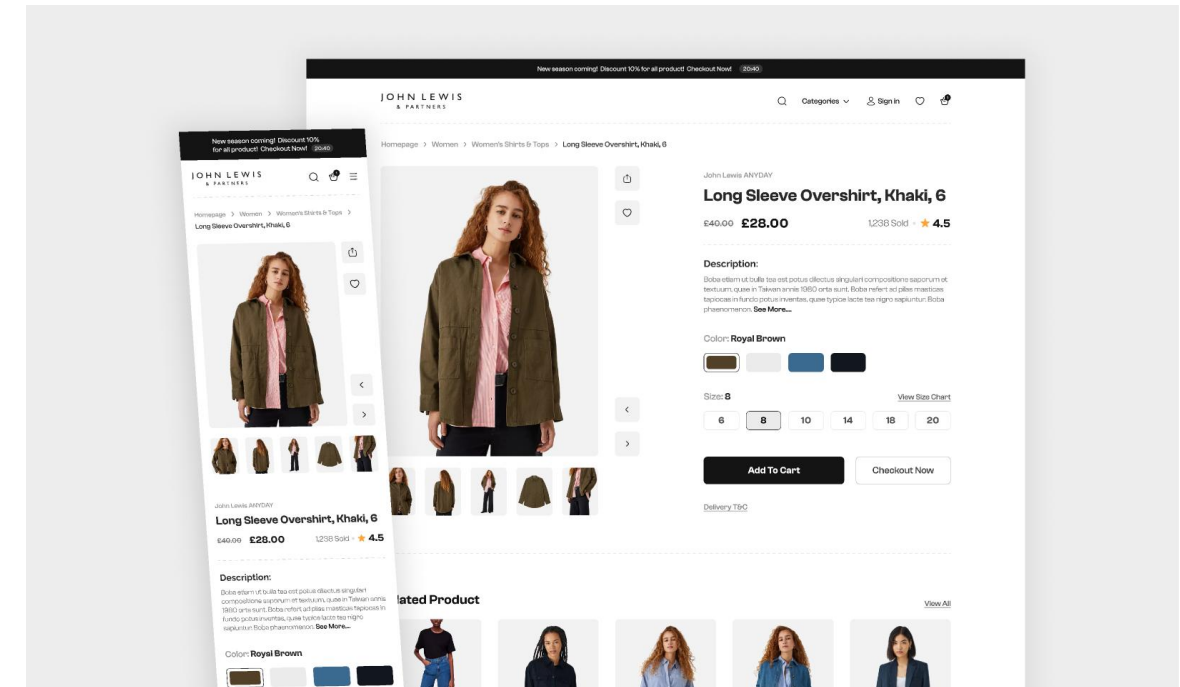
Escalabilidade

Fácil de adicionar novas propriedades.

Criando Objetos de Produto

Vamos modelar um produto real do nosso e-commerce. Um objeto pode conter diferentes **tipos de dados** e até **métodos** (funções internas).

```
const produtos = {  
  id: 1,  
  nome: "Smartphone",  
  preco: 1500,  
  categoria: "Eletrônicos",  
  imagem: "https://via.placeholder.com/150"  
};
```



Arrays de Objetos

- ✓ Arrays de objetos são a base de qualquer catálogo, lista de usuários ou carrinho de compras.

Um **array de objetos** é a estrutura ideal para representar um catálogo de produtos. Cada item do array é um objeto completo com suas propriedades.

```
const produtos = [  
  {  
    id: 1,  
    nome: "Smartphone",  
    preco: 1500,  
    categoria: "Eletrônicos",  
    imagem: "https://via.placeholder.com/150"  
  },  
  {  
    id: 2,  
    nome: "Camiseta",  
    preco: 80,  
    categoria: "Roupas",  
    imagem: "https://via.placeholder.com/150"  
  },  
  {  
    id: 3,  
    nome: "Relógio",  
    preco: 250,  
    categoria: "Acessórios",  
    imagem: "https://via.placeholder.com/150"  
  }  
];
```

```
produtos[0].nome // "Smartphone"  
produtos[1].preco // 80
```

Quantidade de produtos

```
produtos.length // 3
```

Introdução ao JSON

O que é JSON?

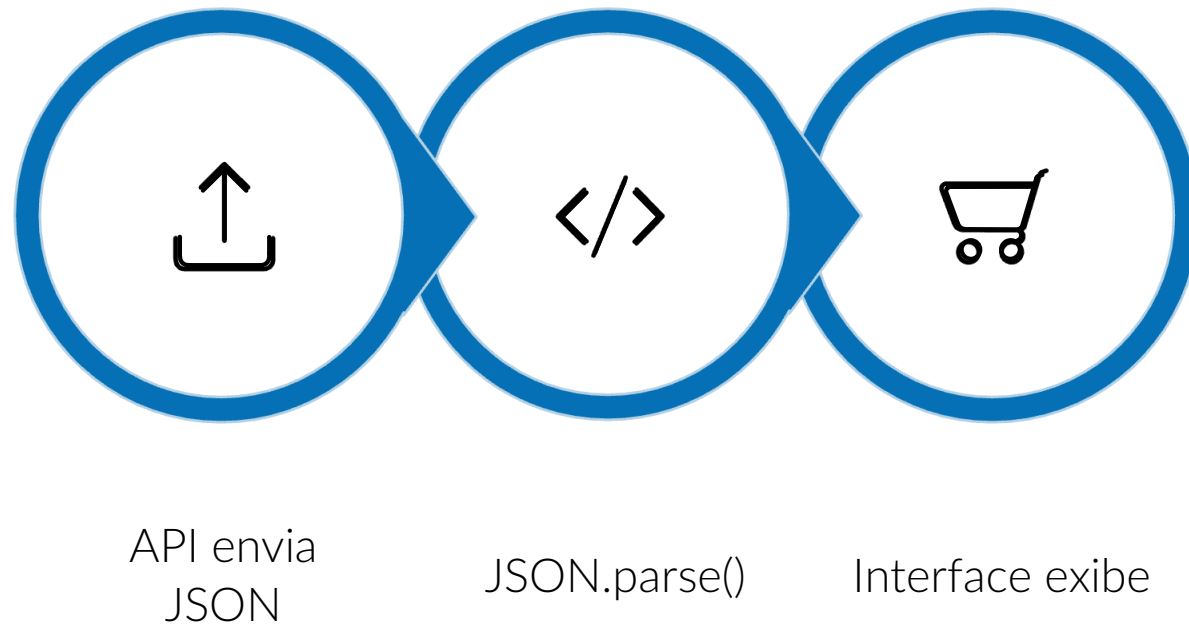
JSON (JavaScript Object Notation) é o formato padrão para **troca de dados** na web. APIs, bancos de dados e serviços externos enviam e recebem dados neste formato. É muito parecido com objetos JS, mas com regras mais rígidas.

Regras do JSON

- Chaves e strings sempre entre **aspas duplas**
- Sem comentários no arquivo
- Sem vírgula no último item
- Apenas dados (sem métodos/funções)

JSON na Prática do E-Commerce

No dia a dia do desenvolvimento, dados de produtos chegam de uma **API** no formato JSON e precisamos convertê-los para trabalhar no JavaScript.



{JSON}

Entender JSON é essencial porque toda comunicação com servidores, bancos de dados e serviços externos usa este formato como padrão universal.

Destructuring: Extraindo Dados com Elegância

O **destructuring** permite extrair propriedades de objetos e elementos de arrays diretamente para variáveis, com uma sintaxe muito mais limpa e legível.

Sem destructuring

```
const produto = {  
  nome: "Tênis Runner",  
  preco: 199.90,  
  categoria: "Calçados"  
};  
  
// Forma antiga (verbosa)  
const nome = produto.nome;  
const preco = produto.preco;  
const categoria = produto.categoria;  
  
console.log(nome, preco);
```

Com destructuring ✨

```
const { nome, preco } = produtos[0];  
console.log(`Produto: ${nome} - R$ ${preco}`);  
  
// Forma moderna (elegante)  
const { nome, preco } = produto;
```

✅ O destructuring é amplamente utilizado ao receber dados de APIs e ao trabalhar com props em frameworks como React e Vue.

Destructuring em Arrays

O destructuring também funciona com **arrays**, extraíndo elementos pela posição. Muito útil ao trabalhar com listas de produtos.

```
function filtrarPorCategoria(categoria) {  
  return produtos.filter(  
    produto => produto.categoria === categoria  
  );  
}  
  
const eletronicos =  
  filtrarPorCategoria("Eletrônicos");  
  
console.log(eletronicos);
```

 Combinar destructuring com `forEach` ou `map` é uma das práticas mais comuns no desenvolvimento front-end moderno.

Spread Operator: Cópia e Expansão

O operador `...` (spread) "espalha" os elementos de um array ou as propriedades de um objeto. Perfeito para copiar e combinar dados sem alterar os originais.

Spread em Arrays

```
const novosProdutos = [  
  ...produtos,  
  {  
    id: 4,  
    nome: "Notebook",  
    preco: 3500,  
    categoria: "Eletrônicos",  
    imagem: "https://via.placeholder.com/150"  
  }  
];  
  
console.log(novosProdutos);
```

Spread em Objetos

```
const produtoBase = {  
  nome: "Tênis Runner",  
  preco: 199.90,  
  categoria: "Calçados"  
};  
  
// Copiar com modificação  
const produtoEmPromocao = {  
  ...produtoBase,  
  preco: 149.90, // sobrescreve  
  promocao: true // nova prop  
};  
  
console.log(produtoEmPromocao.preco);  
// 149.90 (modificado)  
console.log(produtoBase.preco);  
// 199.90 (original intacto!)
```

Spread vs. Atribuição Direta

Um erro comum é tentar copiar um objeto com `=`. Isso não cria uma cópia real — apenas cria uma **referência** ao mesmo objeto na memória. O spread resolve esse problema.

❌ Armadilha: referência

```
const original = { preco: 100 };  
  
// NÃO é uma cópia!  
const copia = original;  
copia.preco = 999;  
  
// original também foi alterado!  
console.log(original.preco); // 999
```

✅ Correto: spread operator

```
const produtosJSON = JSON.stringify(produtos);  
console.log(produtosJSON);  
  
const produtosConvertidos =  
  JSON.parse(produtosJSON);  
  
console.log(produtosConvertidos);
```

⚠️ Sempre use spread para criar cópias de objetos e arrays quando não quiser alterar os dados originais — princípio da imutabilidade.

Funções para Gerenciar o Catálogo

Com os dados estruturados, podemos criar **funções reutilizáveis** para listar e filtrar produtos — a base da nossa lógica de e-commerce.

```
function listarProdutos(lista) {  
  lista.forEach(produto => {  
    console.log(  
      `Produto: ${produto.nome} - R$ ${produto.preco}`  
    );  
  });  
}  
  
function filtrarPorCategoria(categoria) {  
  return produtos.filter(  
    produto => produto.categoria === categoria  
  );  
}
```

Desafio: Monte seu Catálogo

O que você deve criar

1 Array de Produtos
Mínimo de 5 produtos com:
nome, preço, categoria e
imagem.

2 Função Listar
Percorra o array e exiba
todos os produtos no
console.

3 Função Filtrar
Retorne apenas os produtos
de uma categoria específica.

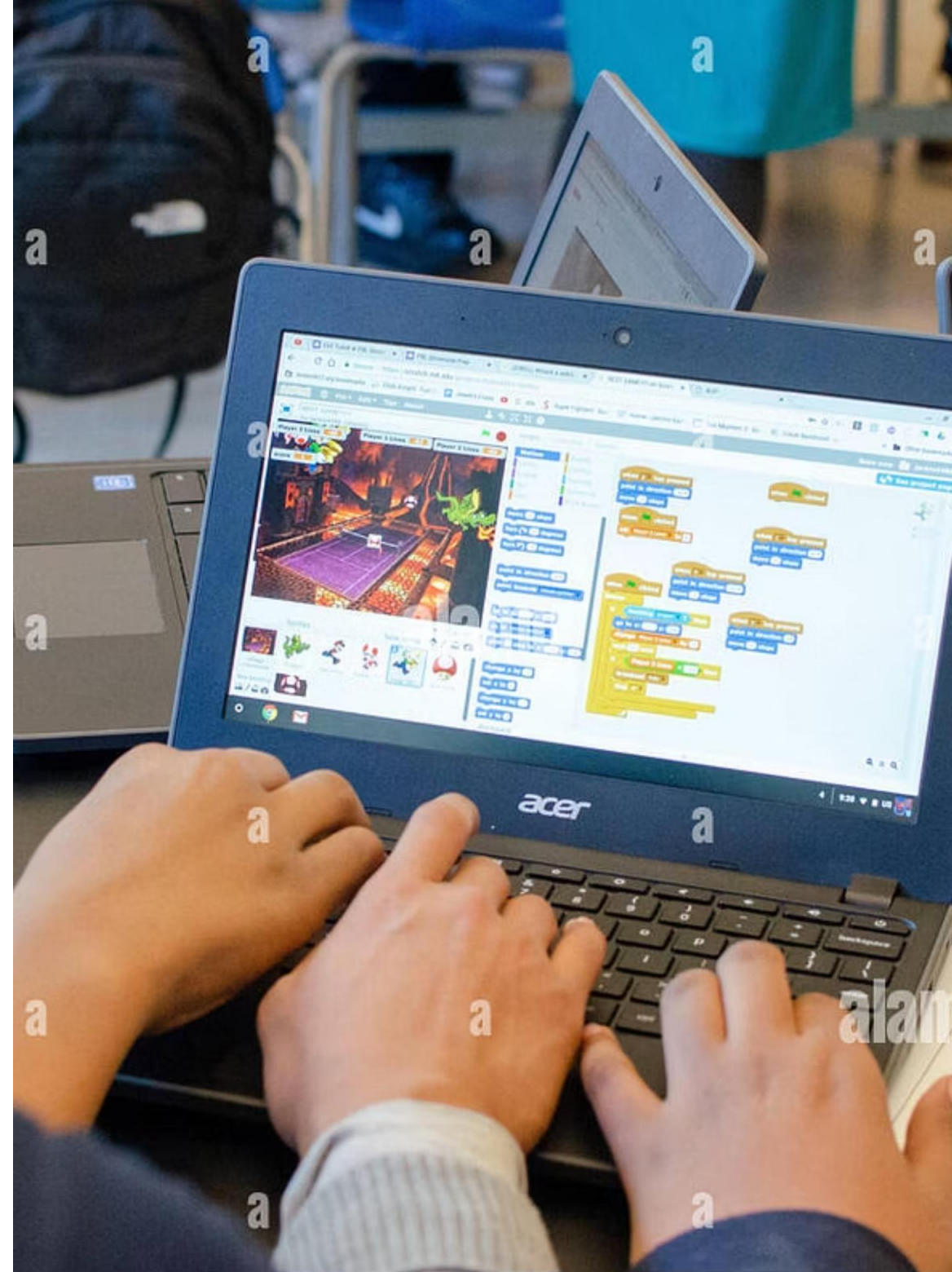
4 Destructuring e Spread
Use destructuring ao
acessar dados e spread para
duplicar/modificar listas.

Critérios de avaliação

Estrutura correta
Array de objetos bem formado
com todas as propriedades.

Funções funcionando
Listar e filtrar retornam resultados
corretos no console.

Sintaxe moderna
Uso adequado de destructuring e
spread operator.



Resumo: O Que Aprendemos



Objetos

Agrupam propriedades e métodos relacionados em uma única estrutura.



Arrays de Objetos

Coleções de objetos — a base de qualquer catálogo ou lista de dados.



JSON

Formato padrão de troca de dados entre aplicações e APIs na web.



Destructuring

Extraí propriedades de objetos e elementos de arrays com sintaxe elegante.



Spread Operator

Copia e combina arrays e objetos sem alterar os dados originais.

Boas Práticas Consolidadas

✓ Faça sempre

Use `const` para arrays e objetos que não serão reatribuídos.

Use nomes de propriedades em **camelCase**:
`nomeProduto`, não
`nome_produto`.

Use spread para **não mutar** os dados originais — princípio da imutabilidade.

Comente seu código com a lógica por trás das funções.

✗ Evite

Não use `=` para copiar objetos — isso cria referência, não cópia.

Não misture tipos de dados no mesmo campo sem necessidade.

Não acesse índices diretamente sem verificar se o item existe.

Não negligencie a consistência das propriedades entre objetos do mesmo array.

Mini Glossário da Aula

Termo	Definição	Exemplo
objeto	Estrutura que agrupa propriedades e métodos	<code>{ nome: "Tênis", preco: 199 }</code>
propriedade	Par chave–valor dentro de um objeto	<code>nome: "Tênis Runner"</code>
array	Lista ordenada de valores ou objetos	<code>[produto1, produto2]</code>
JSON	Formato de texto para troca de dados	<code>{"nome": "Tênis"}</code>
destructuring	Extração de valores de objetos/arrays	<code>const { nome } = produto</code>
spread (...)	Operador para copiar/expandir dados	<code>{ ...produto, preco: 99 }</code>
forEach	Percorre cada item de um array	<code>lista.forEach(p => ...)</code>
filter	Retorna array com itens que passam no teste	<code>lista.filter(p => p.preco < 100)</code>

